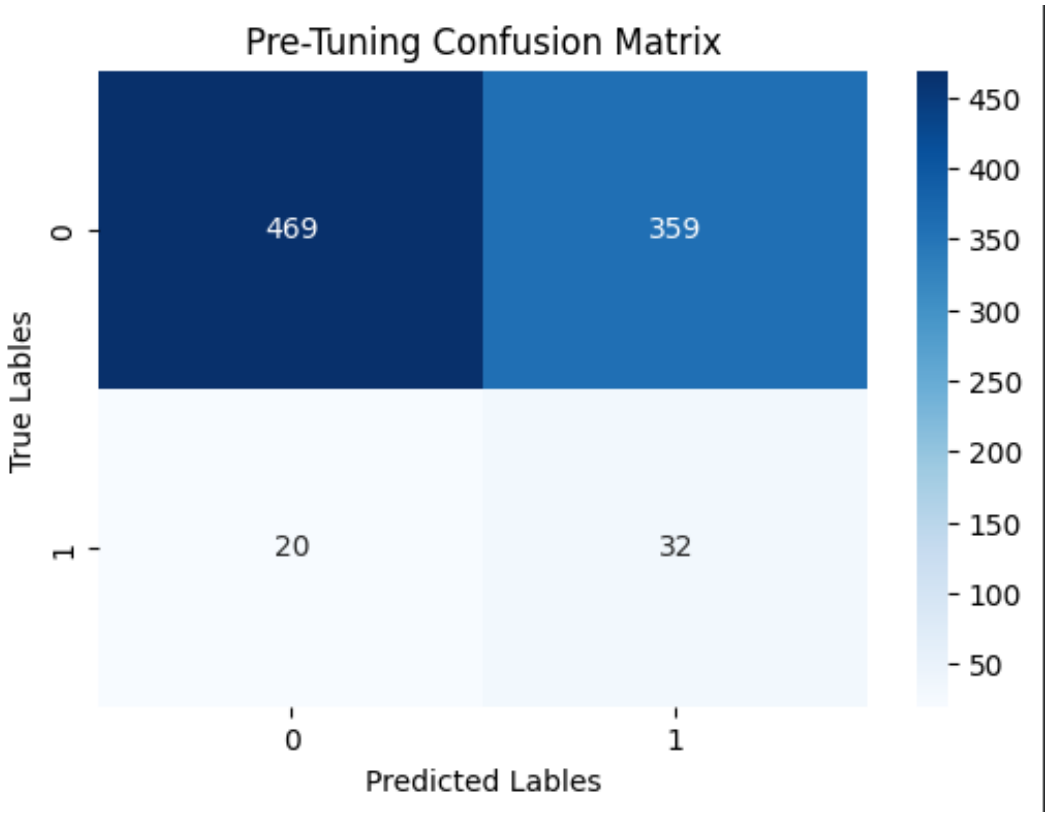
Before discussing the focus of this report, which is the baseline model, I first wanted to highlight changes I have made to the dataset. After getting feedback from the class it was clear I needed to find a way to address multicollinearity present within the columns/features. With this in mind I decided to drop an additional 8 columns from the dataset which were FG, 3P, 2P FT, OWS, DWS, OBPM, and DBPM. These features were all removed because of the direct relation they had to other features and redundancy.

I decided to use logistic regression as the baseline model for my project. Before justifying why think logistic regression is a good fit, I want to take the time to briefly explain how logistic regression works. Logistic regression starts by looking at all the features provided and assigning a value score to each one which indicates the impact each feature has towards the target variable. These individual features values are then combined for each row/player and put through the sigmoid function which lets logistic regression convert any number from 100,000 to –10,000 into a probability between 0 and 1. Finally, logistic regression compares each of the probability values to the model's set threshold to predict whether the player belongs to class 1 or class 0. Now I have three main justifications for why I have chosen logistic regression. My problem or task is binary classification between breakout and non-breakout players. Logistic regression not only naturally but also intuitively functions to predict between class labels using probabilities which is what I want for my project. In addition, GPT agrees that my dataset works well with logistic regression since it is mostly numerical, I will not need to conduct any more preprocessing before running the model. My final reason is due to the simplicity of the model along with the relatively small size of the dataset. I am able to run the model and adjust parameters quickly to establish my baseline before evaluating more complex models.

As I discussed in the second milestone report, I have decided to use the class_weight parameter within logistic regression to handle the class imbalance. I reviewed the scikit learn logistic regression documentation and consulted with GPT on how to properly explain how the class weight parameter impacts the model. What class_weight does is that is calculates a weight value for each of the two classes by taking the total observations divided by the number of classes times specific class observations. This now means that the logistic regression model will now be more sensitive to accurately predicting class 1 in order to reduce the loss with the cost being more errors or mistakes on the majority class.

The next step was to train the logistic regression model with mostly default parameters to serve as a true baseline. I made two changes to the Logistic regression default parameters which I reviewed on scikit learn. Class_weight by default is set to none, but due to the data imbalance present within the data I decided it should be set to 'balanced' for the true baseline. I decided to also increase the max number of iterations (max_iter) to 1000 compared to the default value of 100 to remove the warning lines provided when running the model with default values. The initial results were Accuracy = 0.569, Precision = 0.082, Recall = 0.615, F1_score = 0.144, and the Confusion matrix results which can be seen in the image below.



When conducting the k-fold cross-validation, I decided to consult GPT to help me generate the code since I am unfamiliar with the code used to properly do this. I utilized stratified kfold in order to ensure that when each fold is created the class distribution of split remains the exact same as the original dataset. Due to the relatively small size of the dataset, I decided to use a n_splits or k value of 5. Starting with Accuracy across folds the mean was 0.5670 with a variance value of 0.000133. The Precision mean across folds was 0.1024 and variance being 0.000062. Next was the Recall across folds with a mean of 0.6547 and a variance of 0.003447. The last metric is F1_score with a mean value of 0.1770 and variance of 0.00191 across folds.

In regard to parameter tunning, while I technically tunned a total of 4 logistic regression parameters, I think it is fair to say that I only tunned 2 independent parameters. The first parameter was ‘C’ which represents the opposite of regularization strength. This means that a large C value means the model is not really penalizing large coefficients with small C values meaning the opposite. In this logistic regression model, I was able to get better performance when I increased the value of C meaning that the model is able to perform better when regularization is not strict. The other main parameter I tunned was the penalty which decided whether the model is penalized by ridge, lasso, elasticnet, or none. Penalty has two other parameters that are dependent on what penalty is set to. L1_ratio applies when the penalty is set to elasticnet and decides the ratio of lasso/ridge in the model with the value being closer to 1 being more lasso and 0 being ridge. Solver is the other variable dependent on penalty and is what decides which algorithm to use for optimization. Why I say it is dependent on penalty is because

each algorithm does not work with all of the different penalties; L2 or Ridge works with all 6 penalties which are lbfgs, liblinear, newton-cg, newton-cholesky, sag, and saga. L1 or Lasso works with liblinear and saga. Elasticnet is only able to use saga unless L1_ratio is set to be equal to exactly 0 or 1. After conducting tunning I found that increasing the value of C had a positive impact on the model. It improved the accuracy, precision, recall, and f1-score which can been seen in the table below or found on the python notebook on github.

Comparison	Accuracy	Precision	Recall	F1-score
Before	0.569	0.082	0.615	0.144
After	0.572	0.084	0.635	0.149
Difference	0.003	0.002	0.020	0.005

I consulted with GPT for insights on the topics discussed within the critical analysis. Logistic regression is not a perfect model with there being 3 flaws I think are important to mention. The first flaw is that the model makes the assumption that the relationship is linear when making decisions for predictions. This means that in cases where the relationship between features and the target are nonlinear, the model is almost guaranteed to perform badly since the model is unable to understand a nonlinear relationship with linear decision making. The second limitation is logistic regression's inability to understand interactions between features. While decision trees and XGBoost learn the interactions between features logistic regression treats the features independently which can negatively impact model performance. The final flaw I can speak to based on my own experience is that it seems that logistic regression does not handle data imbalance well. Performance did not improve much even when utilizing class_weight which leads me to conclude I might need to conduct other possible solutions to fixing data imbalance within my model. I will now compare the pros and cons of logistic regression against decision tree. As I said earlier in this paragraph the pros of logistic regression is that is that it is quick, simple to interpret, and regularization parameters to help with overfitting with cons being the forced linear decision making, ignoring feature interactions, and ability to negatively impacted by class imbalance. Now when it comes to decision tree a lot of its strengths are actually the weaknesses of logistic regression. It learns the nonlinear relationships through its branch splitting along with its ability to understand the interactions that happen between features. However, without regularization parameters/penalty decision trees are far more likely to experience overfitting and require more in depth parameter tunning than what I was able to accomplish with my own logistic regression model. There are two improvements I would like to make for future milestones. The first is to address the data imbalance with an alternative, like resampling to try and improve feature model(s) performance. The second goal I have is to use a model like decision tree or random forest to analyze the nonlinear relationship and feature interactions.